FINAL REPORT

Team 10 - Cal Club Field Hockey

Katie Sundstrom (CEO), Alaina Kolling (CCO), Ryan Stevenson, Jason Suh,
Alex Tan, Max Shih, Joseph Saldivar, Paari Thanikachalam







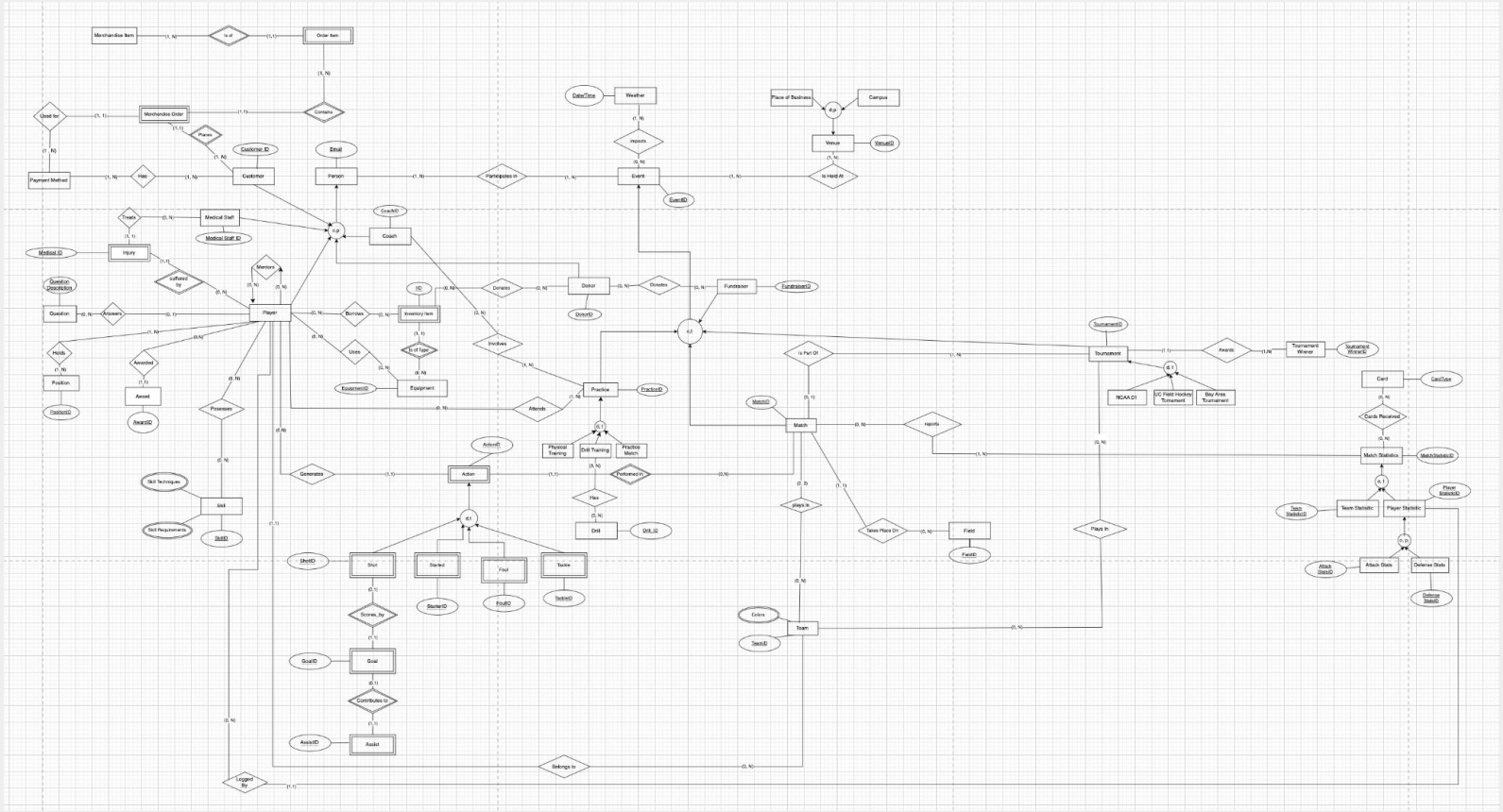
Client Introduction

The Cal Field Hockey Club is an official student sports organization at UC Berkeley. Led by UC Berkeley students and created for UC Berkeley students, this club team offers the opportunity of participating in field hockey practices, games, and team events. The Cal Field Hockey Club hosts weekly practices at the fields above the Upper Hearst Structure located on the northern end of the UC Berkeley campus. The club team competes against other field hockey club teams located in Northern California, such as: UC Davis, Santa Clara University, and Cal Poly San Luis Obispo.

Previously, the club team collected gameday-related and scheduling-related data on paper (based on memory) and then transferred it over to be kept safely on a Google Sheet. The team collected player-related data through an interest form that would then automatically populate a Google Sheet. With the creation of this database, we are transforming the way the club collects, organizes, maintains, and analyzes their data. With it, the club has the potential to operate more efficiently and improve their skills through effective analysis of collected data.

Client Contact:
Cal Field Hockey Club
Leadership Team: Aly Turfler, Maddy Kester
(858) 900-8525
alyturf@berkeley.edu, mekester@berkeley.edu

Simplified EER Diagram



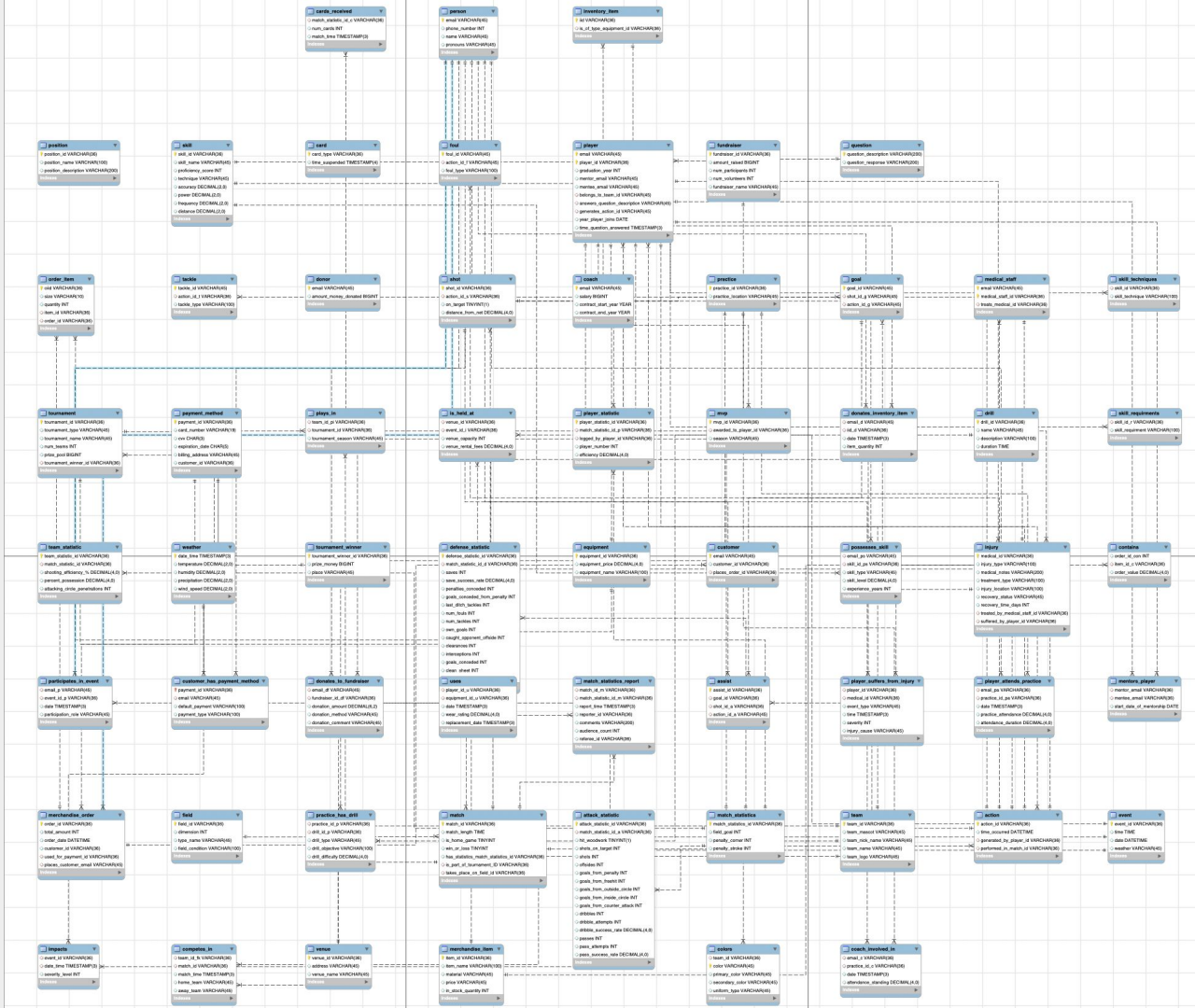
Relational Design Schema

1. Person(Email, Phone Number, Name, Pronouns)
2. Player(Player ID, Email[1], Year, Mentor Email, Mentee Email, Belongs_to_TeamID[30], Year_Player_Joins, Answers_QuestionDescription[28], Time_Question_Answered)
3. Coach(Coach_ID, Email[1], Salary, Contracted Start Year, Contracted End Year)
4. Customer(Customer_ID, Email[1], Places_OrderID[17])
5. Medical_Staff(Medical Staff ID, Email[1], Treats_MedicalID[21])
6. Donor(Donor_ID, Email[1], Amount_of_Money_Donated)
7. Event(EventID, Time, Date, Weather)
8. Match(MatchID, EventID[7], Match_Length, Is_Home_Game, Win_Or_Loss, Is_Part_Of_TournamentID[12], Takes_Place_On_FieldID[14])
9. Fundraiser(FundraiserID, EventID[7], Amount Raised, Number of Participants, Number of Volunteers, Fundraising Name)
10. Practice(PracticeID, EventID[7], Practice_Duration)
11. Drill(Drill_ID, Name, Description)
12. Tournament(TournamentID, EventID[7], Tournament_Type, Tournament_Name, Number_of_Teams, Prize_Pool, TournamentWinnerID[13])
13. Tournament Winner(TournamentWinnerID, Prize Money, Place)
14. Field(FieldID, Dimension, TypeName, FieldCondition)
15. Weather(Date/Time,Temperature, Humidity, Precipitation, Wind Speed)
16. Venue(VenueID, Address, VenueName)
17. Merchandise_Order(Order_ID, Total Amount, Order Date, Customer ID, Used_For_Payment_ID[18], Places_Customer_Email[1])
18. Payment_Method(Payment ID, Card Number, CVV, Expiration Date, Billing Address, Customer ID)
19. Order Item(OIID, Size, Quantity, Item ID[20], Order ID[17])
20. Merchandise Item(Item ID, Item Name, Material, Price, In Stock Quantity)
21. Injury(Medical_ID, Injury_Type, Medical_Notes, Treatment_Type, Injury_location, Recovery_Status, Recovery_Time, treats_Medical_Staff_ID[5], suffered_by_PlayerID[2])
22. Skill(SkillID, SkillName, Proficiency_Score, Accuracy, Power, Distance, Frequency)
23. Action(ActionID, TimeOccurred, Generated_by_PlayerID[2], Performed_in_MatchID[8])
24. Goal(GoalID, ShotID[39], ActionID[23])
25. Foul(FoulID, ActionID[23], Foul_Type)
26. Tackle(TackleID, ActionID[23], Tackle_Type, Success)
27. Assist(AssistID, GoalID[24],ShotID[39], ActionID[23])
28. Question(Question Description, Question Response)
29. Position(PositionID, PositionName, Position_Description)
30. Team(TeamID, Team Mascot, Team Nick Name, Team Name, Logo)
31. Match Statistics(MatchStatisticsID, Field Goal, Penalty Corner, Penalty Stroke)
32. Team Statistic(TeamStatisticID, MatchStatisticsID[31], Shooting Efficiency (%), Percent Possession, Attacking Circle Penetrations)
33. Player Statistic(PlayerStatisticID, MatchStatisticsID[31], Logged_by_PlayerID[2], Player Number, Efficiency)

33. Player Statistic(PlayerStatisticID, MatchStatisticsID[31], Logged_by_PlayerID[2], Player Number, Efficiency)

34. Attack Statistic(AttackStatisticID, MatchStatisticsID[31], Hit Woodwork, Shots On Target, Shots, Offsides, Goals From Penalty, Goals From Freehit, Goals From Outside Circle, Goals From Inside Circle, Goals From Counter Attack, Dribbles, Dribble Attempts, Dribble Success Rate, Passes, Pass Attempts, Pass Success Rate)
35. Defense Statistic(DefenseStatisticID, MatchStatisticsID[31], Saves, Save Success Rate, Penalties Conceded, Goals Conceded From Penalty, Last-ditch Tackles, Number of Fouls, Number of Tackles, Own Goals, Caught Opponent Offside, Clearances, Interceptions, Goals Conceded, Clean Sheet, Blocks)
36. Card(CardType, Time Suspended)
37. Equipment(EquipmentID, Equipment Price, Equipment Name)
38. Inventory_Item(IID, is_of_type_EquipmentID[37])
39. Shot(ShotID, ActionID[23], OnTarget, Distance_From_Net)
40. Impacts(Event_ID[7],Date/Time[15], Severity_Level)
41. Is_Held_At(Venue_ID[16], Event_ID[7], Venue_Capacity, Venue_Rental_Fees)
42. Contains(Order_ID[17], Item_ID[20], Order_Value)*
43. Customer_Has_Payment_Method(Payment_ID[18], Email[1], Default_Payment, Payment_Type)
44. Skill_Techniques(SkillID[22], Skill_Technique)
45. Skill_Requirements(SkillID[22], Skill_Requirement)*
46. Player_Suffers_From_Injury(PlayerID[2], Medical_ID[21], Event_Type, Time, Severity, Injury_Cause)
47. Colors(TeamID[30], Color, Primary_Color, Secondary_Color, Uniform_Type)
48. Competes_In (TeamID[30], Match ID[8], Match_Time, Home_Team, Away_Team)
49. Plays_In(TeamID[30], Tournament ID[12], Tournament_Season)
50. Cards_Received(MatchStatisticID[31], Number of Cards, Match_Time)
51. Uses(PlayerID[2], EquipmentID[37], Date, Wear_Rating, Replacement_Date)
52. Practice_Has_Drill(PracticeID[10], Drill_ID[11], Drill_Type, Drill_Objective, Drill_Difficulty)
53. Participates_in_Event(Email[1], EventID[7], Date, Participation_Role)
54. Coach_Involved_In(Email[1], PracticeID[10], Date, Attendance_Standing)
55. Possesses_Skill(Email[1], SkillID[22], SkillType, Skill_Level, Experience_Years)
56. Player_Attends_Practice(Email[1], PracticeID[10], Date, Practice_Attendance, Attendance_Duration)
57. Donates_Inventory_Item(Email[1], IID[38], Date, Item_Quantity)
58. Donates_to_Fundraiser(Email[1], FundraiserID[9], Donation_Amount, Donation_Method, Donation_Comment)
59. Mentors_Player(Mentor_Email[2], Mentee_Email[2], Start_date_of_mentorship)
60. Award(MVPID, Awarded_to_PlayerID[2], Season)
61. Match_Statistics_Reports_Match(MatchID[8], MatchStatisticID[31], Report_Time, ReporterID, Comments, Audience_Count, RefereeID)
62. Started(SAID, ActionID[23],Regular_Starter, Lineup_Order)

MySQL Workbench



Interesting Query 1

Interesting Query 1

Goal: Create a ranking system for all the players on the team

Benefits for the client: Able to use the rankings to have a non bias ranking system which can be used to give awards like mvp along with giving insight to who might the team want to start in future games.

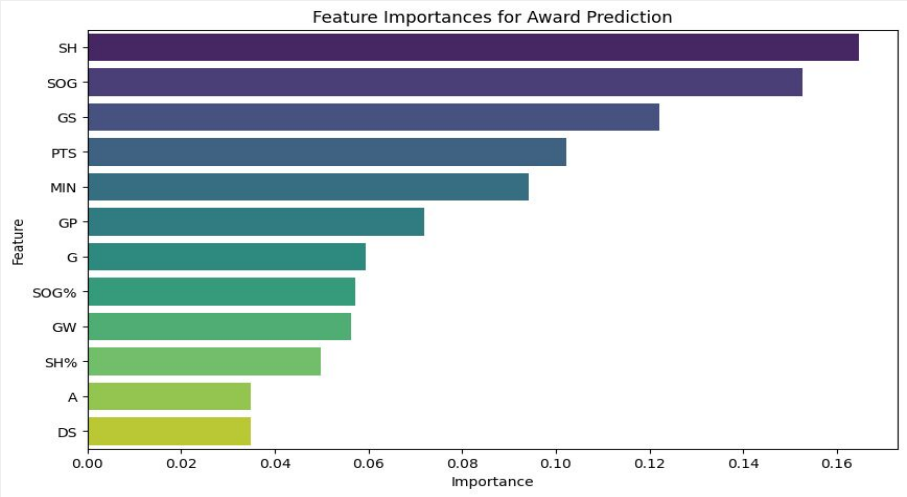
Process: Use historical player stats data and awards from previous years to train a ML model that finds what stats contribute to a player getting an award. The SQL query will extract the player statistics for current players and will create a ranking metric using the weights from the ML model.

Interesting Query 1: Assumptions and Model

To rank the players in terms of their various stats we choose to find what stats that a player generates contributes most to them getting awards/accolades.

Using historical data we fit a Random Forest Model with the target feature being awards received to derive the Feature Importances for Award Prediction

This allowed us to get accurate numbers to use for the weights of our player score instead of arbitrarily placing weights on them



Interesting Query 1: Synthetic Data

Synthetic data was created for the Person, Player, Action, Assist, Goal, Shot, Started, and Match tables

For the Person and Player data we used Faker which is what pydbgen uses to generate all the player names, emails, phone numbers, and just gave the players ids starting from 1

```
fake = Faker()
Faker.seed(2)

#players and matches in a season
num_players = 20
num_matches = 20

#make player data
person_data = pd.DataFrame({
    "Name": [fake.name() for _ in range(num_players)],
    "Phone Number": [fake.phone_number() for _ in range(num_players)],
    "Email": [fake.email() for _ in range(num_players)],
    "Pronouns": np.random.choice(["They/Them", "She/Her"], size=num_players),
})

#make playerids
person_data['PlayerID'] = [f"P{i}" for i in range(num_players)]

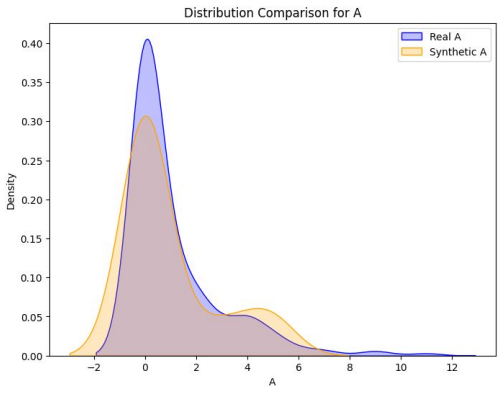
#add year and team
player_data = pd.DataFrame({
    "PlayerID": person_data["PlayerID"],
    "Email": person_data["Email"],
    "Year": np.random.choice([2024], size=num_players),
    "Belongs_to_TeamID": np.random.choice(["T1"], size=num_players)
})
```

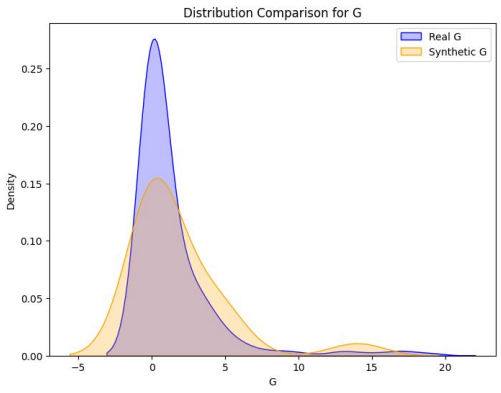
Interesting Query 1: Synthetic Data Cont.

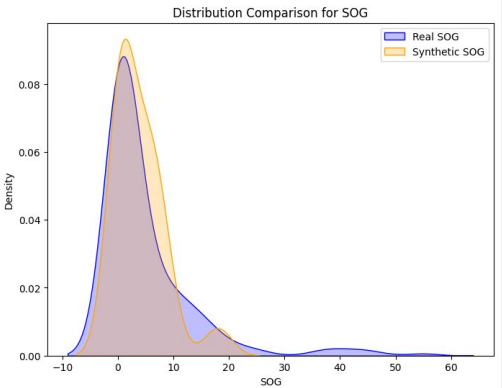
The rest of the tables were trickier since using random distributions wouldn't give realistic stats due to a lot of logical constraints on the data

We instead created the synthetic stats from distributions fit on the historical player stats along with lots of logical checks

To ensure realistic results we compared distributions from the synthetic stats and real ones along with seeing if the stats are equally correlated between datasets







Interesting Query 1: Tables in MySQL Workbench

person

email	phone_number	name	pronouns
aaron78@example.net	001-674-857-2547x8767	Stephen Yoder	They/Them
alexandriarodriguez@example.net	629-547-4488x88996	Theresa Brown	She/Her
edwardsalexandra@example.com	6238130877	Jordan Lutz	She/Her
ethornton@example.com	001-282-504-0222x173	Fernando James	They/Them
fischertodd@example.net	327.394.8619	Keith Knapp	They/Them
gavin27@example.net	001-282-343-3060x05523	Elizabeth Obrien	She/Her

player

email	player_id	graduation_ye...
alexandriarodriguez@example.net	P0	2024
hdavis@example.org	P1	2024
edwardsalexandra@example.com	P10	2024
lisasmith@example.org	P11	2024
gwhitehead@example.com	P12	2024

shot

shot_id	action_id_s
S05	A8
S100	A122
S101	A125
S102	A126
S103	A127
S104	A128
S105	A130
S112	A138

goal

goal_id	shot_id_g	action_id_g
G05	S28	A36
G06	S45	A55
G07	S47	A59
G09	S74	A89
G10	S78	A95
G11	S84	A102
G12	S89	A108
G13	S91	A111
G15	S100	A123

action

action_id	time_occurr...	generated_by_player...	performed_in_match...
A100	NULL	P2	M18
A101	NULL	P2	M19
A102	NULL	P2	M19
A103	NULL	P2	M20
A107	NULL	P5	M12
A108	NULL	P5	M12
A109	NULL	P5	M15
A110	NULL	P5	M15
A111	NULL	P5	M15
A112	NULL	P5	M16

match

match_id
M09
M1
M10
M11
M12
M13
M14

assist

assist_id	goal_id	action_id_a
A10	G15	A124
A14	G26	A225
A15	G27	A231
A16	G28	A238
A5	G05	A37
A6	G06	A56

Interesting Query 1: SQL Code

```
SELECT
    player.player_id,
    person.name,
    COUNT(DISTINCT shot.shot_id) AS Total_Shots,
    COUNT(DISTINCT StarterActions.performed_in_match_id) AS Total_Games_Started,
    COUNT(DISTINCT goal.goal_id) AS Total_Goals,
    COUNT(DISTINCT assist.assist_id) AS Total_Assists
FROM player
JOIN person ON player.email = person.email
LEFT JOIN actions ON player.player_id = actions.generated_by_player_id
LEFT JOIN goal ON actions.action_id = goal.action_id_g
LEFT JOIN shot ON actions.action_id = shot.action_id_s
LEFT JOIN assist ON actions.action_id = assist.action_id_a
LEFT JOIN started ON actions.action_id = started.action_id_st
LEFT JOIN actions AS StarterActions ON started.action_id_st = StarterActions.action_id
GROUP BY player.player_id, person.name
```

Interesting Query 1: Output in MySQL Workbench

```
1   SELECT
2       player.player_id,
3       person.name,
4       COUNT(DISTINCT shot.shot_id) AS Total_Shots,
5       COUNT(DISTINCT StarterActions.performed_in_match_id) AS Total_Games_Started,
6       COUNT(DISTINCT goal.goal_id) AS Total_Goals,
7       COUNT(DISTINCT assist.assist_id) AS Total_Assists
8   FROM player
9   JOIN person ON player.email = person.email
10  LEFT JOIN actions ON player.player_id = actions.generated_by_player_id
11  LEFT JOIN goal ON actions.action_id = goal.action_id_g
12  LEFT JOIN shot ON actions.action_id = shot.action_id_s
13  LEFT JOIN assist ON actions.action_id = assist.action_id_a
14  LEFT JOIN started ON actions.action_id = started.action_id_st
15  LEFT JOIN actions AS StarterActions ON started.action_id_st = StarterActions.action_id
16  GROUP BY player.player_id, person.name
```

player_id	name	Total_Shots	Total_Games_Start...	Total_Goals	Total_Assists
P0	Theresa Brown	1	7	0	0
P1	Russell Fitzgerald	23	11	3	2
P10	Jordan Lutz	9	11	1	0
P11	Catherine English	8	1	1	0
P12	Morgan Knox	0	10	0	0
P13	Rachel Gomez	3	1	1	0
P14	Dr. Candace Webster	13	8	0	0
P15	Julia Moreno	20	11	3	3
P16	Tim Mason PhD	19	11	3	0
P17	Stephen Yoder	1	9	0	0
P18	Keith Knapp	10	1	0	0
P19	Katelyn Walker	3	2	0	0
P2	Elizabeth Obrien	15	11	3	1
P3	Lauren Moore	0	1	0	0
P4	James Sharp	0	10	0	0
P5	Tiffany Stephens MD	7	1	2	0
P6	Andrew Johnson	7	7	2	1
P7	Fernando James	1	1	0	0
P8	Kelly Castro	0	1	0	0
P9	Alexander Nguyen	0	1	0	0

Interesting Query 1: Non-SQL Formulas

After the SQL data extraction, the data is inputted into a pandas dataframe in python to create additional features like SH%, PTS

	PlayerID	Player	SH	GS	G	A
0	P0	Theresa Brown	5	15	2	2
1	P1	Russell Fitzgerald	44	18	5	4
2	P10	Jordan Lutz	15	15	2	0
3	P11	Catherine English	17	7	4	1
4	P12	Morgan Knox	0	16	0	0

	PlayerID	Player	SH	GS	G	A	PTS	SH%
0	P0	Theresa Brown	5	15	2	2	19	0.400000
1	P1	Russell Fitzgerald	44	18	5	4	27	0.113636
2	P10	Jordan Lutz	15	15	2	0	17	0.133333
3	P11	Catherine English	17	7	4	1	12	0.235294
4	P12	Morgan Knox	0	16	0	0	16	0.000000

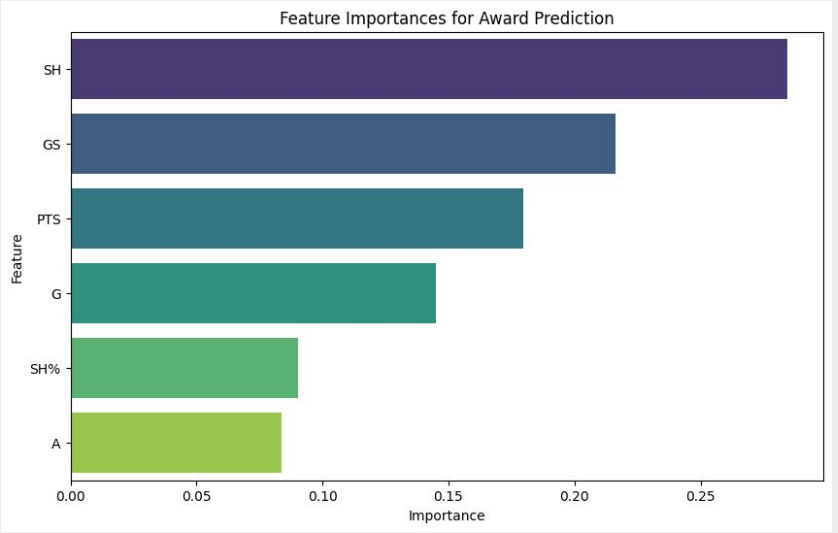
These stats are then ready to be passed through the weighted score formula to give players a score to rank them by

Interesting Query 1: Non-SQL Formulas

Weighted_Score = (w1 * SH+ w2 * GS + w3 * PTS + w4 * G + w5 * SH% + w6 * A)

Where w1-w6 correspond to the weights we derived from the feature importance of the RandomForestModel

	PlayerID	Player	SH	GS	G	A	PTS	SH%	Weighted_Score	Rank
1	P1	Russell Fitzgerald	44	18	5	4	27	0.113636	22.332983	1
7	P15	Julia Moreno	39	18	4	4	26	0.102564	20.585334	2
12	P2	Elizabeth Obrien	36	18	4	2	24	0.111111	19.205847	3
8	P16	Tim Mason PhD	33	18	5	1	24	0.151515	18.417872	4
6	P14	Dr. Candace Webster	19	15	1	1	17	0.052632	11.939545	5
2	P10	Jordan Lutz	15	15	2	0	17	0.133333	10.870882	6
16	P6	Andrew Johnson	9	13	3	2	18	0.333333	9.242730	7
3	P11	Catherine English	17	7	4	1	12	0.235294	9.192317	8
0	P0	Theresa Brown	5	15	2	2	19	0.400000	8.578955	9
9	P17	Stephen Yoder	2	16	0	0	16	0.000000	6.908804	10



Interesting Query 2

Interesting Query 2: Description

Determine optimal starting lineup of outfield players for the upcoming match by considering their performance and allow for rotation by resting the players who have played most already.

We automatically disregard the 3 players who have played the most matches so far.

We then choose 3 defenders, 3 midfielders, and 3 attackers.

- The three defenders with the most tackles will be chosen to start.
- The three midfielders with the most assists will be chosen to start.
- The three attackers with the most goals will be chosen to start.

From here, we can we can predict the number of goals scored by the selected team by combining each players' scoring rates.

We can verify this methodology by going through old matches, combining the players' scoring rates, and comparing this value to the true number of goals scored by the team in that match.

Interesting Query 2: Synthetic Data

Normalized distribution of goals, tackles, assists from Actions.

Higher number of tackles for defensive players, goals and assists for offensive players

Distribution of player positions considered; less goalies than midfielders.

Not all tackles are successful (60%)

PlayerID	MatchID	TackleID	Success
1	4	T_1_4_0	TRUE
1	1	T_1_1_0	FALSE
2	6	T_2_6_0	TRUE
3	3	T_3_3_0	TRUE
3	11	T_3_11_0	TRUE
3	20	T_3_20_0	FALSE
3	19	T_3_19_0	TRUE
4	7	T_4_7_0	TRUE
4	11	T_4_11_0	FALSE
5	11	T_5_11_0	FALSE

```python
def generate_goals(self):
    """
    generate goal data with bias towards forwards
    """
    goals = []
    for _, player_match in self.player_matches.iterrows():
        player_id = player_match['PlayerID']
        match_id = player_match['MatchID']

        # Assign goal probabilities based on player position
        position = self.players.loc[self.players['PlayerID']
        goal_probs = {
            'Forward': 0.6,
            'Midfielder': 0.3,
            'Defender': 0.1,
            'Goalkeeper': 0.0
```

Interesting Query 2: Tables in MySQL Workbench

player

email	player_id	graduation_ye...
alexandriarodriguez@example.net	P0	2024
hdavis@example.org	P1	2024
edwardsalexandra@example.com	P10	2024
lisasmith@example.org	P11	2024
gwhitehead@example.com	P12	2024

tackle

tackle_id	action_id_t	success
TID1	AT0	1
TID10	AT9	0
TID11	AT10	1
TID12	AT11	0
TID13	AT12	1
TID14	AT13	0

shot

shot_id	action_id_s
S05	A8
S100	A122
S101	A125
S102	A126
S103	A127
S104	A128
S105	A130
S112	A138

goal

goal_id	shot_id_g	action_id_g
G05	S28	A36
G06	S45	A55
G07	S47	A59
G09	S74	A89
G10	S78	A95
G11	S84	A102
G12	S89	A108
G13	S91	A111
G15	S100	A123

action

action_id	time_occurr...	generated_by_player...	performed_in_match...
A100	NULL	P2	M18
A101	NULL	P2	M19
A102	NULL	P2	M19
A103	NULL	P2	M20
A107	NULL	P5	M12
A108	NULL	P5	M12
A109	NULL	P5	M15
A110	NULL	P5	M15
A111	NULL	P5	M15
A112	NULL	P5	M16

match

match_id
M09
M1
M10
M11
M12
M13
M14

assist

assist_id	goal_id	action_id_a
A10	G15	A124
A14	G26	A225
A15	G27	A231
A16	G28	A238
A5	G05	A37
A6	G06	A56

Interesting Query 2: SQL Code: Selecting Lineup

```
WITH PlayerMatchCounts AS (
    SELECT
        actions.generated_by_player_id AS PlayerID,
        COUNT(DISTINCT actions.performed_in_match_id) AS MatchesPlayed
    FROM actions
    GROUP BY actions.generated_by_player_id
),

FilteredPlayers AS (
    SELECT PlayerID
    FROM PlayerMatchCounts
    ORDER BY MatchesPlayed DESC
    LIMIT 18446744073709551615 OFFSET 3
),

PerformanceMetrics AS (
    SELECT
        player.player_id AS PlayerID,
        COUNT(DISTINCT tackle.tackle_id) AS Tackles,
        SUM(CASE WHEN tackle.success = 1 THEN 1 ELSE 0 END) AS SuccessfulTackles,
        COUNT(DISTINCT assist.assist_id) AS Assists,
        COUNT(DISTINCT goal.goal_id) AS Goals,
        pmc.MatchesPlayed
    FROM player
    LEFT JOIN actions ON player.player_id = actions.generated_by_player_id
    LEFT JOIN tackle ON actions.action_id = tackle.action_id_t
    LEFT JOIN shot ON actions.action_id = shot.action_id_s
    LEFT JOIN goal ON shot.shot_id = goal.shot_id_g
    LEFT JOIN assist ON goal.goal_id = assist.goal_id
    LEFT JOIN PlayerMatchCounts pmc ON player.player_id = pmc.PlayerID
    WHERE player.player_id IN (SELECT PlayerID FROM FilteredPlayers)
    GROUP BY player.player_id, pmc.MatchesPlayed
),

TopTackles AS (
    SELECT PlayerID
    FROM PerformanceMetrics
    ORDER BY Tackles DESC
    LIMIT 3
),
TopAssists AS (
    SELECT PlayerID
    FROM PerformanceMetrics
    WHERE PlayerID NOT IN (SELECT PlayerID FROM TopTackles)
    ORDER BY Assists DESC
    LIMIT 3
),
TopGoals AS (
    SELECT PlayerID
    FROM PerformanceMetrics
    WHERE PlayerID NOT IN (
        SELECT PlayerID FROM TopTackles
        UNION
        SELECT PlayerID FROM TopAssists
    )
    ORDER BY Goals DESC
    LIMIT 3
),
TopPlayers AS (
    SELECT PlayerID FROM TopTackles
    UNION ALL
    SELECT PlayerID FROM TopAssists
    UNION ALL
    SELECT PlayerID FROM TopGoals
)
SELECT
    pm.PlayerID,
    pm.MatchesPlayed,
    pm.Tackles,
    pm.SuccessfulTackles,
    pm.Assists,
    pm.Goals
FROM PerformanceMetrics pm
WHERE pm.PlayerID IN (SELECT PlayerID FROM TopPlayers);
```

Interesting Query 2: Output in MySQL Workbench

```
1
2   WITH PlayerMatchCounts AS (
3       SELECT
4           actions.generated_by_player_id AS PlayerID,
5           COUNT(DISTINCT actions.performed_in_match_id) AS MatchesPlayed
6       FROM actions
7       GROUP BY actions.generated_by_player_id
8   ),
9   FilteredPlayers AS (
10      SELECT PlayerID
11      FROM PlayerMatchCounts
12      ORDER BY MatchesPlayed DESC
13      LIMIT 18446744073709551615 OFFSET 3 -- Skip the top 3 players
14  ),
15  PerformanceMetrics AS (
16      SELECT
17          player.player_id AS PlayerID,
18          COUNT(DISTINCT tackle.tackle_id) AS Tackles,
```

PlayerID	MatchesPlayed	Tackles	SuccessfulTackl...	Assists	Goals
P0	16	3	0	0	0
P8	7	3	0	0	0
P18	16	3	2	0	0
P15	18	3	2	3	3
P6	15	2	2	1	2
P10	16	0	0	0	1
P5	10	2	1	0	2
P11	13	2	1	0	1

Interesting Query 2: Non-SQL Formulas

With the new table of 9 outfield players, we can predict the number of goals scored by that team by combining each of their scoring rates.

```
starters.loc[:, "Goals per Game"] = starters["Goals"] / starters["MatchesPlayed"]
starters
```

✓ 0.0s

	PlayerID	MatchesPlayed	Tackles	Assists	Goals	Successful Tackles	Goals per Game
2	3	18	4.0	2.0	1.0	3	0.055556
5	6	19	2.0	3.0	3.0	1	0.157895
6	7	18	1.0	4.0	2.0	1	0.111111
7	8	19	3.0	5.0	1.0	2	0.052632
24	25	16	1.0	4.0	3.0	1	0.187500
26	27	16	4.0	0.0	0.0	3	0.000000
27	28	16	0.0	5.0	2.0	0	0.125000
28	29	12	4.0	0.0	1.0	4	0.083333
29	30	12	1.0	1.0	3.0	1	0.250000

```
print("Expected goals scored with this lineup: ", sum(starters["Goals per Game"]))
```

✓ 0.0s

```
Expected goals scored with this lineup:  1.023026315789474
```

As we can see, based on the starting lineup and their historic scoring patterns, we can expect that the lineup chosen by our algorithm will score roughly 1 goal in their next game, suggesting that the team we're working with is not very high scoring.

Normalization Analysis

Without modifying our relations, here some that are not in 2NF, 3NF, or BCNF & how we would fix them:

1. Payment_Method (Payment ID, Card Number, CVV, Expiration Date, Billing Address, Customer ID)
    a. Not in BCNF since Customer ID → Street, City, State, Zip Code while Customer ID is not a superkey.
        i. **Fixed:** Payment_Method (Payment ID, Customer ID, Card Number, CVV, Expiration Date)
        Customer(Customer ID, Street, City, State, Zip Code)
2. Participates_in_Event(Email[1], EventID[7], Date, Participation_Role)
    a. Not in 2NF since EventID → Time, Date, Weather, so Date is not wholly dependent on the entire key.
        i. **Fixed:** Participates_in_Event(Email[1], EventID[7], Participation_Role)
3. Coach_Involved_In(Email[1], PracticeID[10], Date, Attendance_Standing)
    a. Not in 3NF since PracticeID → EventID → Time, Date, Weather shows transitive dependencies, so taking out Date breaks the transitiveness apart from the current relation.
        i. **Fixed:**
        Coach_Involved_In(Email[1], PracticeID[10], Attendance_Standing)
        Practice(PracticeID, EventID[7], Practice_Duration)
        Event(EventID, Time, Date, Weather)

After modifying our relations, here are some that would violate 1NF, 2NF, 3NF, or BCNF & how we would fix them:
* It is important to note that our relations did not originally have these violations, we just added them

1. Skill(SkillID, SkillName, Proficiency_Score, Accuracy, Power, Distance, Frequency, Techniques, Requirements)
    a. Since Techniques and Requirements are multi-valued attributes, this relation violates 1NF. Therefore, they must be separated into their own relations.
        i. **Fixed:**
        Skill(SkillID, SkillName, Proficiency_Score, Accuracy, Power, Distance, Frequency)
        Skill_Techniques(SkillID[22], Skill_Technique)
        Skill_Requirements(SkillID[22], Skill_Requirement)
2. Team(TeamID, Team Mascot, Team Nick Name, Team Name, Logo, Colors)
    a. Since Colors is a multi-valued attribute, this relation violates 1NF. Therefore, it must be separated into its own relation.
        i. **Fixed:**
        Team(TeamID, Team Mascot, Team Nick Name, Team Name, Logo)
        Colors(TeamID[30], Color, Primary_Color, Secondary_Color, Uniform_Type)

Normalization Analysis: 1NF → 2NF

***2NF** disallows partial dependencies. A partial dependency occurs when a non-prime attribute depends on only a portion of the primary key.*

Before Normalization:

a. Participates_in_Event(Email[1], EventID[7], Date, Participation_Role)
   i. This is not in 2NF since EventID → Time, Date, Weather. Therefore, Date is not wholly dependent on the entire key as it doesn't need Email as part of its primary key.

After Normalization:

b. Participates_in_Event(Email[1], EventID[7], Participation_Role)
c. Event(EventID, Time, Date, Weather)
   i. This is now in 2NF since we eliminated the partial dependency, and Date is still accessible through EventID.

Normalization Analysis: 2NF → 3NF

***3NF** disallows transitive dependencies. A transitive dependency occurs when a non-prime attribute depends on another non-prime attribute.*

Before Normalization:

a. Coach_Involved_In(Email[1], PracticeID[10], EventID, Attendance_Standing, Date)
    i. This is not in 3NF since PracticeID → EventID → Time, Date, Weather shows transitive dependencies.

After Normalization:

a. Coach_Involved_In(Email[1], PracticeID[10], Attendance_Standing)
b. Practice(PracticeID, EventID[7], Practice_Duration)
c. Event(EventID, Time, Date, Weather)
    i. This is now in 3NF as Date can be retrieved by following PracticeID → EventID → Date through schemas a, b, c instead of all in a.

Normalization Analysis: 3NF → BCNF

Boyce Codd Normal Form (BCNF) *states that for every non-trivial functional dependency X → Y, X is a superkey. It ensures that all partial dependencies and transitive dependencies are eliminated.*

Before Normalization:

a. Payment_Method (Payment ID, Card Number, CVV, Expiration Date, Billing Address, Customer ID)
    i. This is not in BCNF since Customer ID → Street, City, State, Zip Code while Customer ID is not a superkey.

After Normalization:

a. Payment_Method (Payment ID, Customer ID, Card Number, CVV, Expiration Date)
b. Customer_Address(Customer ID, Street, City, State, Zip Code)
    a. This is now in BCNF since Street, City, State, Zip Code can now be retrieved from CustomerID which is a superkey.

Team Member Contributions

Alaina Kolling (CCO): scheduled team meetings, submitted assignments, worked on eer design (mostly merchandise + person sections), contributed to converting eer model into relations, implemented database in MySQL Workbench, created 62 tables, imported synthetic data into tables, executed query to get final output tables for our 2 queries, exported data from MySQL workbench, worked on query 2, attended and presented in each DP review

Joseph Saldivar: Drafted the original query 1 and the final updated query 1, generated all the synthetic data used in query one, wrote both the SQL and non-SQL formulas for query one, Presented in DP 2 and Final presentation, worked on the EER model building the action action entity and all its subclasses, contributed to converting the eer model into relations, modified synthetic data to be compatible in MySQL Workbench

Max Shih: EER diagram, Relational Schema, DP1 Presentation, DP2 presentation

Ryan Stevenson: worked on EER diagram (mostly event, types of events, and player statistics), contributed to converting EER model into relations , came up with final query 2 + original query 2 and query 3, wrote non-SQL formulas for original and final query 2, helped write SQL queries for original and final query 2, attended and presented in each DP review

Jason Suh: EER diagram, Relational Schema, DP2 Presentation, Query 2 SQL, Query 2 Synthetic Data, BCNF Normalization

Katie Sundstrom (CEO): maintained communication with client (met with them to discuss ideas, kept them in the loop, and invited them to final presentation), wrote team's Client Proposal, wrote Client Introduction, contributed to EER diagram, converted our original Advanced EER diagram to a simplified EER diagram, contributed to converting EER model into Relations, did final review of Relations, did Normalization Analysis, attended and presented in each DP review

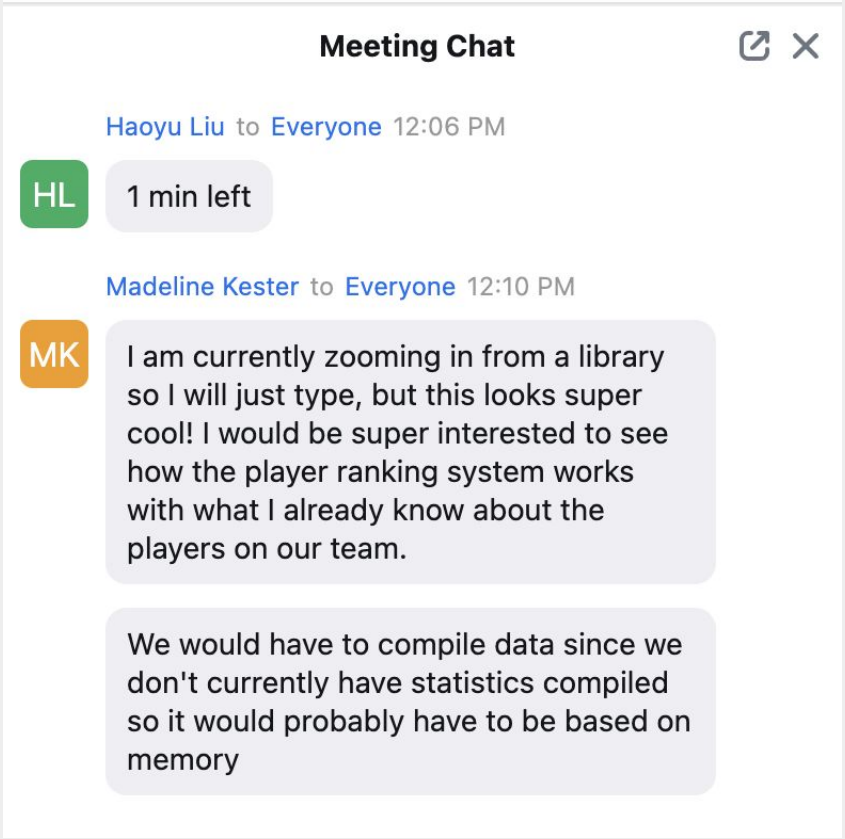
Alex Tan: worked on EER diagram (mostly on match statistics, player roles, and quality checking), contributed converting EER to relations and simplifying the relations post DP2. Assisted in checking and proofreading the 2 SQL queries and synthetic data creation. Performed normalization on 2NF, 3NF, and BCNF and presented the findings in the DP reviews.

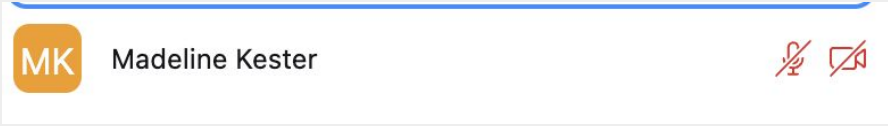
Paari Thanikachalam: who?

Discussion & Future Work

- Through our project, we have successfully implemented:
  - A database prototype for the Cal Club Field Hockey to Track the following Major categories of data:
    - Players (and all related performance / injury / role status / relations to other players)
    - Events/Matches (and all related match statistics / fundraisers / date time for all current event types)
    - Inventories/Merchandise (Including all club equipment / transactions)
    - And More
  - Several sample queries that showcase how the database design can be used for various analysis including:
    - Determining a player's chance of becoming MVP of the season
    - Determining the optimal lineup for a match
  - A series of database normalization so that there are no 2NF, 3NF, and BCNF violations in the prototype:
- Some future work we may extend our analysis to include:
  - Work with the Cal Club Field Hockey to run our queries with their most up-to-date data to perform preliminary analysis.
  - Currently our database schemas supports many operations and records many statistics of the club players at Cal. In the future, we could integrate our current metric with the Cal Field Hockey team to compare player statistics for outstanding players able to play for larger leagues.
  - We have currently constructed several queries that supports our model's evaluation of the best lineups/ players under our weights. In the future, we could use the same metrics to cross compare professional leagues to evaluate the accuracy of our models, and to determine if more data queries should be added to enhance the current method.

Confirmation that Client Attended Final Presentation





Maddy Kester, member of the Cal Field Hockey Leadership Team, attended our Final Presentation.